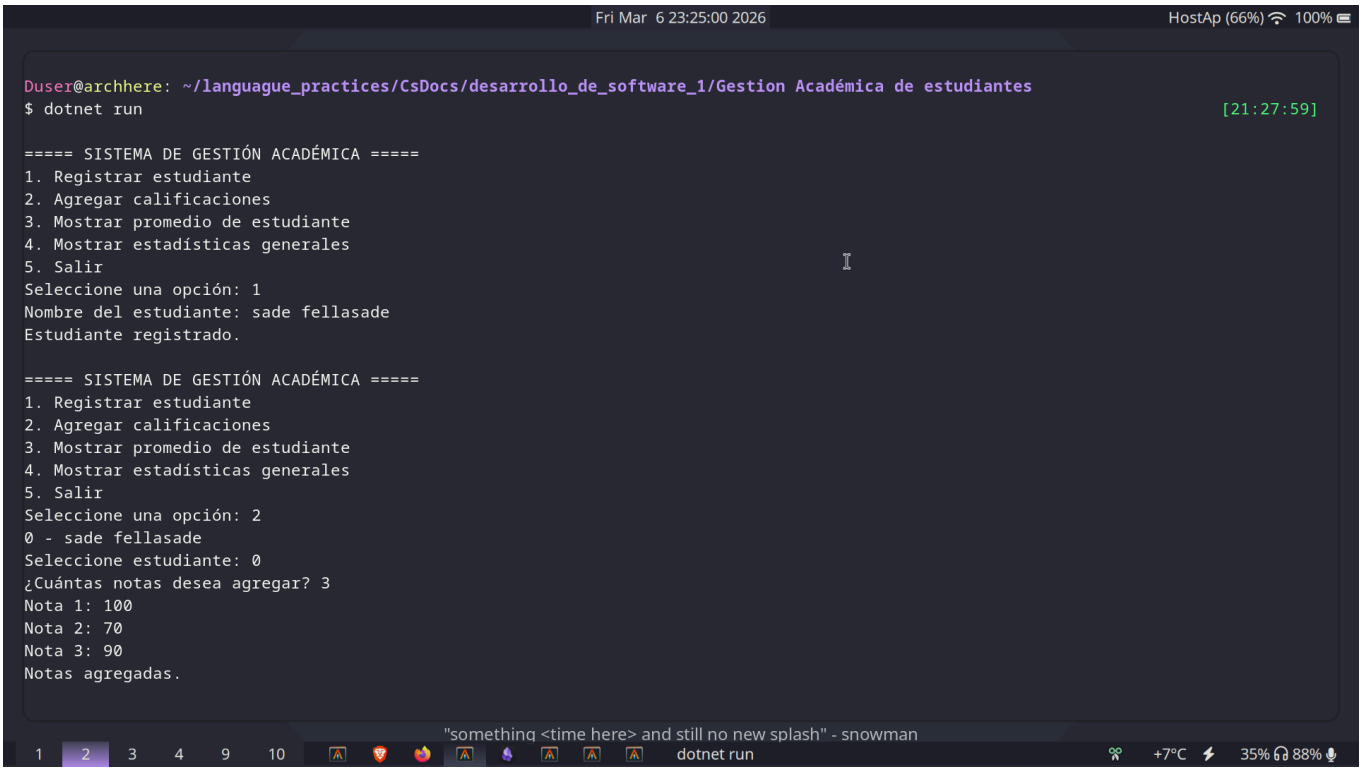
Sistema de gestion de calificaciones

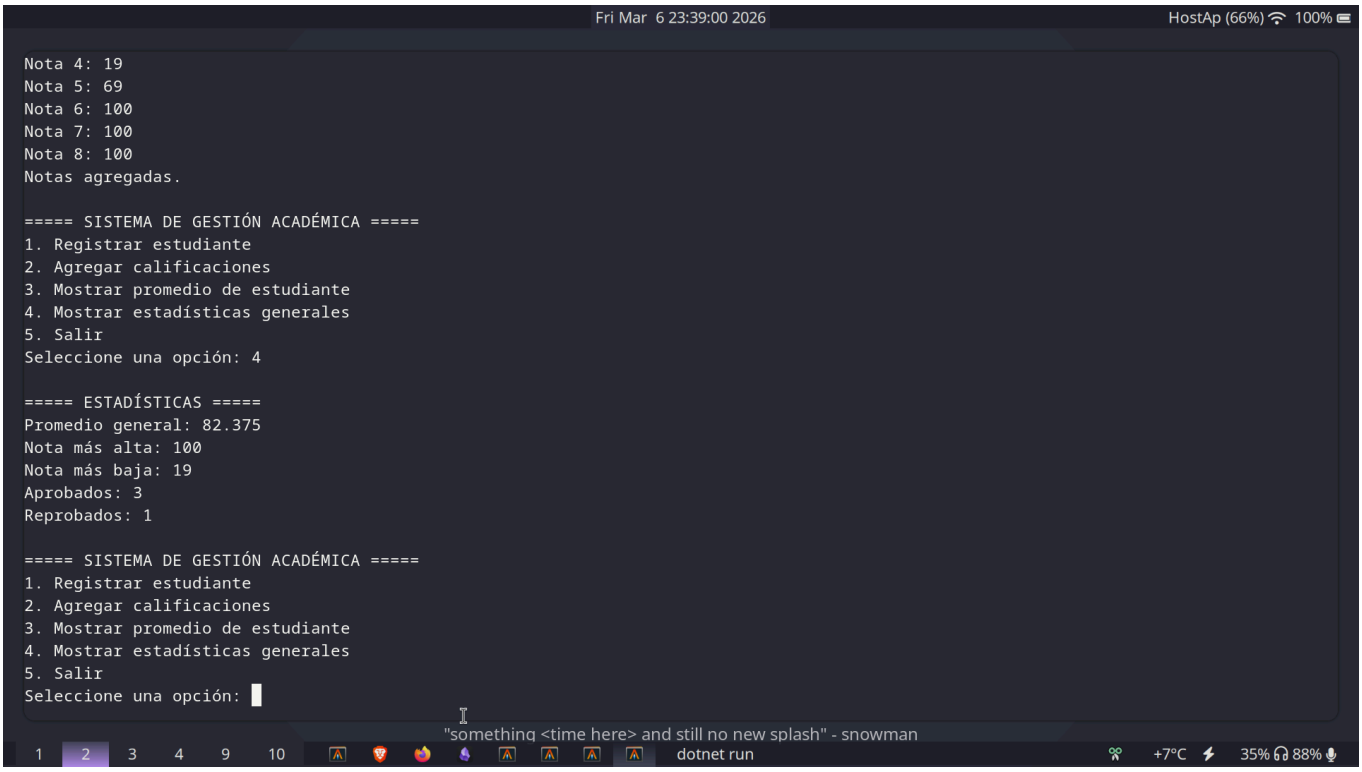


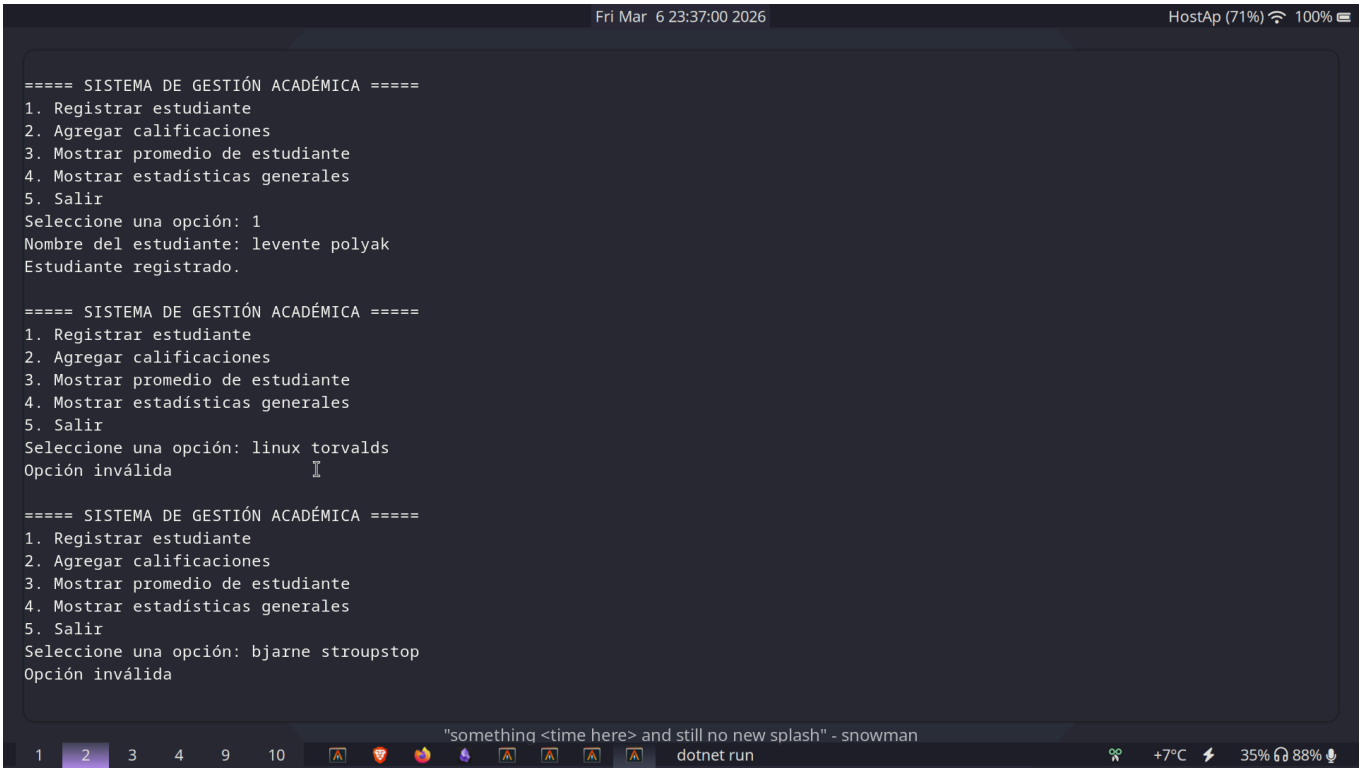
Descripción del problema

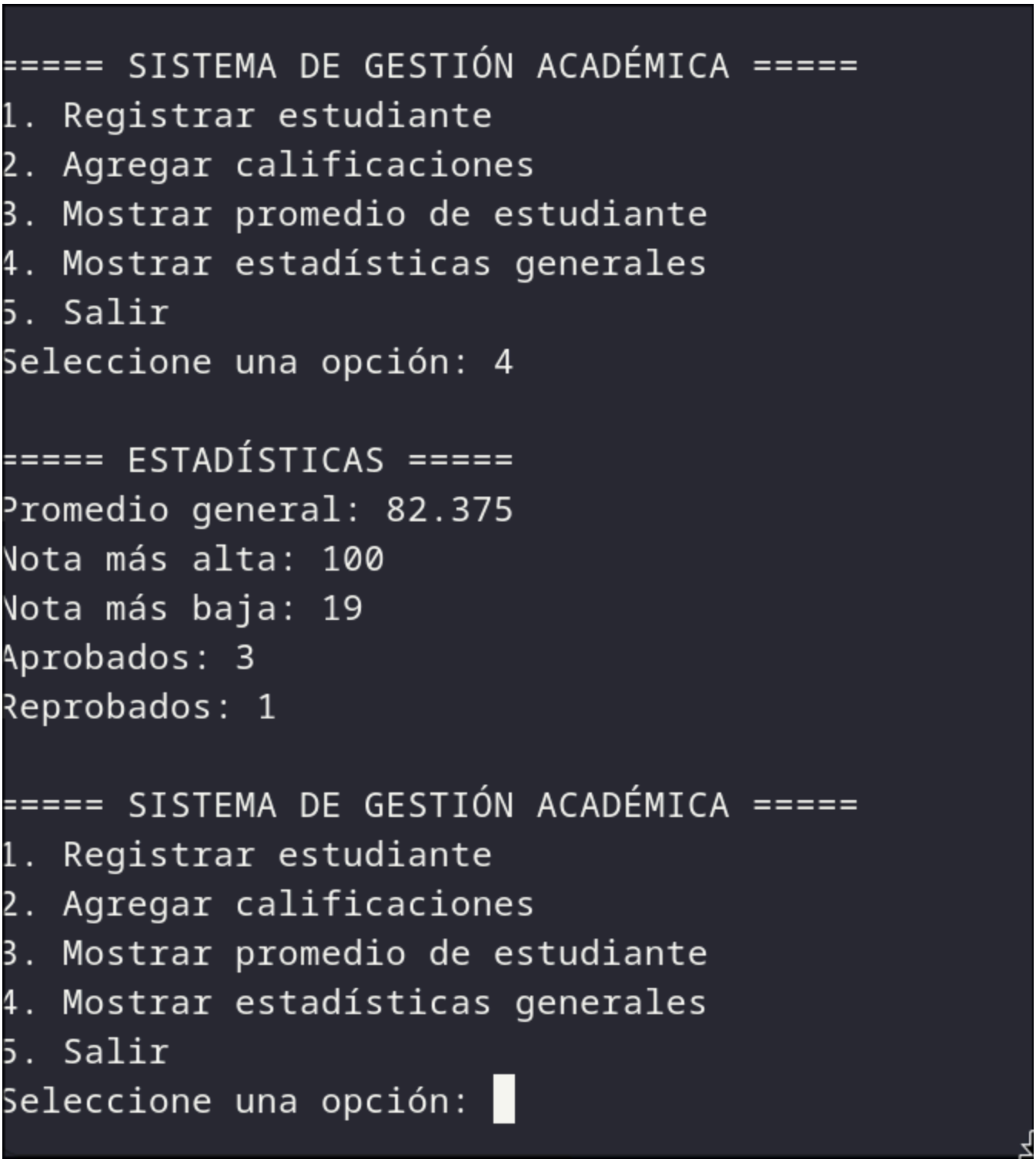
Este sistema resuelve el problema de la gestión de calificaciones de forma manual. El sistema se apodera de la computación, lo cual permite el manejo de la información más eficiente que la version manual. El sistema permite la inserción de datos como el nombre y las notas de los estudiantes, realizando analisis de datos sencillos pero eficaces

Capturas del sistema funcionando


Fri Mar 6 23:25:00 2026
HostAp (66%) 100%
Duser@archhere: ~/languague_practices/CsDocs/desarrollo_de_software_1/Gestion Académica de estudiantes
$ dotnet run
[21:27:59]
===== SISTEMA DE GESTIÓN ACADÉMICA =====
1. Registrar estudiante
2. Agregar calificaciones
3. Mostrar promedio de estudiante
4. Mostrar estadísticas generales
5. Salir
Seleccione una opción: 1
Nombre del estudiante: sade fellasade
Estudiante registrado.
===== SISTEMA DE GESTIÓN ACADÉMICA =====
1. Registrar estudiante
2. Agregar calificaciones
3. Mostrar promedio de estudiante
4. Mostrar estadísticas generales
5. Salir
Seleccione una opción: 2
0 - sade fellasade
Seleccione estudiante: 0
¿Cuántas notas desea agregar? 3
Nota 1: 100
Nota 2: 70
Nota 3: 90
Notas agregadas.
"something <time here> and still no new splash" - snowman
1 2 3 4 9 10
dotnet run
+7°C
35% 88%


Fri Mar 6 23:39:00 2026
HostAp (66%) 100%
Nota 4: 19
Nota 5: 69
Nota 6: 100
Nota 7: 100
Nota 8: 100
Notas agregadas.
===== SISTEMA DE GESTIÓN ACADÉMICA =====
1. Registrar estudiante
2. Agregar calificaciones
3. Mostrar promedio de estudiante
4. Mostrar estadísticas generales
5. Salir
Seleccione una opción: 4
===== ESTADÍSTICAS =====
Promedio general: 82.375
Nota más alta: 100
Nota más baja: 19
Aprobados: 3
Reprobados: 1
===== SISTEMA DE GESTIÓN ACADÉMICA =====
1. Registrar estudiante
2. Agregar calificaciones
3. Mostrar promedio de estudiante
4. Mostrar estadísticas generales
5. Salir
Seleccione una opción:
"something <time here> and still no new splash" - snowman
1 2 3 4 9 10
dotnet run
+7°C
35% 88%


Fri Mar 6 23:37:00 2026
HostAp (71%) 100%
===== SISTEMA DE GESTIÓN ACADÉMICA =====
1. Registrar estudiante
2. Agregar calificaciones
3. Mostrar promedio de estudiante
4. Mostrar estadísticas generales
5. Salir
Seleccione una opción: 1
Nombre del estudiante: levente polyak
Estudiante registrado.
===== SISTEMA DE GESTIÓN ACADÉMICA =====
1. Registrar estudiante
2. Agregar calificaciones
3. Mostrar promedio de estudiante
4. Mostrar estadísticas generales
5. Salir
Seleccione una opción: linux torvalds
Opción inválida
===== SISTEMA DE GESTIÓN ACADÉMICA =====
1. Registrar estudiante
2. Agregar calificaciones
3. Mostrar promedio de estudiante
4. Mostrar estadísticas generales
5. Salir
Seleccione una opción: bjarne stroupstop
Opción inválida
"something <time here> and still no new splash" - snowman
1 2 3 4 9 10
dotnet run
+7°C
35% 88%



Explicación de estructuras de control

Las estructuras de control son expresiones que permiten el control estructurado de bloques de código. Estas estructuras pueden ser bucles(for, foreach, while, do-while), los cuales permiten repetir bloques de Código de forma específica y controlada; o condicionales(if, else if, switch) que permiten ejecutar bloques de código siempre y cuando se cumpla o no una condición.

En el código fuente, ambas estructuras se utilizan para controlar el flojo de expresiones(Código fuente).

Explicación de uso de arreglos y bucles

Los **arreglos y bucles** son herramientas muy importantes.
principalmente, **los arreglos son estructuras de datos** que se almacenan contiguamente en la memoria, lo cual permite el acceso rapido a los datos por esta misma de propiedad amigable con la cache.
Los **bucles** por otro lado, son expresiones de estructuras de control que permiten la repetición de bloques de código de forma estructurada.

En el código fuente, se utilizan listas dinámicas basadas en los arreglos, lo cual aprovecha la velocidad de acceso de los arrays pero lo combina con la el dinamismo de poder agrandar el espacio contiguo.

Además se utilizan diversos tipos de bucles, tanto para manterner corriendo de forma indefinida el programa, como para realizar analisis de datos

Explicación de funciones implementadas

Funciones implementadas

- **Main()**
  Es la función principal. Utiliza el bucle principal, el cual llama las otras funciones utilizando condiciones específicas
  No recibe ningun ni retorna ningun parámetro
- **MostrarMenu()**
  Tal como dice el nombre Muestra el menu de opciones disponibles para el programa.

  Esta es la primera función llamada por la funcion Main una vez se realiza el bucle principal.

  No recibe ningun ni retorna ningun parámetro
- **RegistrarEstudiante()**
  Esta funcion se utiliza tal como dice el nombre, para registrar el estudiante en la lista. En este caso, se almacena el nombre del estudiante en una lista principal llamada estudiantes.

  La función es llamada por la función Main después de que se hace un bloque condicional, cuando la condicion es 1, se ejecuta la funcion RegistrarEstudiante()
- **ComprobarEstudiante()**
  Esta función nace de la ultima refactorización. Esto era un código que se utilizaba en agregarNotas() y Mostrar promedio que hacía el código más largo.

  Esta funcion verifica que la cantidad de estudiantes se diferente de 0. De serlo, Muestra todas los estudiantes disponibles.
  ademas, solicita al usuario insertar un numero de indice para seleccionar al estudiante

adecuado.
Despues de esto, revisa que indice sea correcto.

Esta función es llamada por agregarNotas() y Y mostrar promedio()

No recibe ningun ni retorna ningun parámetro

- **agregarNotas()**
  Esta función agrega notas a la lista de notas disponible y pide la cantidad de notas a agregar. Primero llama la función comprobar Estudiante; Solicita la Cantidad de notas a insertar y utiliza un bucle para insertar todas las notas que el usuario necesite y las agrega a la lista de notas.

  La función es llamada por la función Main después de que se hace un bloque condicional, cuando la condicion es 2, se ejecuta la funcion agregarNotas()

  No recibe ningun ni retorna ningun parámetro
- **MostrarPromedio()**
  Esta funcion muestra el promedio de un estudiante. Primero llama la funcion comprobar estudiante para evitar errores, y una vez terminada, utiliza la función promedio y la muestra.

  La función es llamada por la función Main después de que se hace un bloque condicional, cuando la condicion es 3, se ejecuta la funcion Mostrar promedio()

  No recibe ningun ni retorna ningun parámetro
- **CalcularPromedio()**
- Esta funcion Realiza el proceso de promedio.
  La funcion itera sobre las notas de un estudiante, mientras suma el total de las notas y luego divide el total entre la cantidad de notas.
  la funcion es llamada por las funciones: Mostrar promedio y mostrar estadisticas

  esta función recibe un entero y devuelve un numero de punto flotante con doble precisión
- **MostrarEstadisticas()**
  Esta función realiza las estadisticas del trabajo.
  llama promedios, revisa maximos, minimos y cuenta personas aprobadas.

  La función es llamada por la función Main después de que se hace un bloque condicional, cuando la condicion es 3, se ejecuta la funcion Mostrar promedio()

  No recibe ningun ni retorna ningun parámetro
- **Evidencia de refactorización**

Durante el desarrollo del sistema de gestión académica se realizaron algunas mejoras en el código con el objetivo de hacerlo **más claro, organizado y fácil de mantener**.

Una de las mejoras principales fue **dividir el programa en diferentes funciones**, en lugar de colocar toda la lógica dentro del método `Main`. En el código se implementaron funciones como:

- `MostrarMenu()`
- `RegistrarEstudiante()`
- `AgregarNotas()`
- `MostrarPromedio()`
- `CalcularPromedio()`
- `MostrarEstadisticas()`

Esta separación permite que cada función tenga una **responsabilidad específica**, lo que mejora la legibilidad del código.

Por ejemplo, el cálculo del promedio de un estudiante se movió a una función independiente llamada `CalcularPromedio(int indiceNota)`. Esto evita repetir el mismo algoritmo en varias partes del programa y permite reutilizar la lógica cuando sea necesario.

También se eliminaron repeticiones de código mediante el uso de **bucles** para recorrer las listas de estudiantes y sus calificaciones. Esto hace que el programa sea más eficiente y fácil de modificar si en el futuro se agregan nuevas funcionalidades.

Además, se utilizaron **nombres de variables descriptivos**, como `estudiantes`, `notas`, `promedioGeneral`, `notaAlta` y `notaBaja`, lo que facilita la comprensión del propósito de cada variable dentro del sistema.

Otra mejora importante fue el uso de **validaciones**, por ejemplo al verificar si existen estudiantes registrados antes de agregar notas o calcular promedios. Esto evita errores durante la ejecución del programa.

En conclusión, la refactorización permitió que el sistema tenga una **estructura más modular, clara y reutilizable**, facilitando tanto su comprensión como su mantenimiento.

- ## Respuestas del quiz
  ### 1. ¿Cuál es la diferencia entre función y método?
  Una **función** es un bloque de código reutilizable que realiza una tarea específica y puede devolver un valor. Generalmente es independiente y se puede utilizar en distintos contextos dentro de un programa.

Un **método** es una función que pertenece a una **clase u objeto** dentro de la programación orientada a objetos. Los métodos pueden acceder y modificar los datos del objeto al que pertenecen.

2. ¿Qué ventajas tiene modularizar el código?

Modularizar el código consiste en dividir un programa en **funciones o módulos más pequeños**, cada uno con una responsabilidad específica.

Las principales ventajas son:

- **Mejora la organización del código**, haciéndolo más fácil de leer.
- **Facilita el mantenimiento**, ya que los cambios se realizan en partes específicas del programa.
- **Permite reutilizar funciones** en diferentes partes del sistema.
- **Reduce la duplicación de código**.
- **Facilita la detección y corrección de errores**.

3. ¿Qué problemas genera no usar funciones?

No utilizar funciones provoca varios problemas en el desarrollo del software:

- El código se vuelve **muy largo y difícil de entender**.
- Se produce **duplicación de código**, lo que aumenta el riesgo de errores.
- Es **más difícil mantener y modificar** el programa.
- La depuración de errores se vuelve **más complicada**.
- El programa pierde **claridad y organización**.

- ## Conclusión personal
  El desarrollo de este sistema de gestión académica permitió aplicar varios conceptos fundamentales de programación vistos durante las unidades III, IV y V de la asignatura de Desarrollo de Software.

Durante la realización de la práctica se utilizaron **estructuras de control** como `if`, `else` y `switch` para manejar las diferentes opciones del menú y validar las acciones del usuario. Esto permitió crear una aplicación interactiva que responde a las decisiones del usuario dentro del sistema.

También se implementó el uso de **arreglos y listas** para almacenar información sobre los estudiantes y sus calificaciones. A través del uso de **bucles como `for` y `foreach`**, fue posible recorrer estas estructuras de datos para realizar cálculos como el promedio de notas, la

identificación de la nota más alta y la nota más baja, así como el conteo de estudiantes aprobados y reprobados.

Otro aspecto importante fue la utilización de **funciones y métodos** para dividir el programa en partes más pequeñas y organizadas. Esto permitió mejorar la estructura del código, facilitando su lectura, mantenimiento y reutilización. Cada función se encargó de realizar una tarea específica dentro del sistema, como registrar estudiantes, agregar notas o mostrar estadísticas.

Además, se aplicó el concepto de **refactorización**, con el objetivo de mejorar la calidad del código sin modificar su funcionamiento. Esto ayudó a eliminar código repetido y a organizar mejor la lógica del programa.

Esta práctica permitió comprender mejor cómo se integran diferentes conceptos de programación para desarrollar una aplicación funcional. También permitió fortalecer habilidades relacionadas con la organización del código, la lógica de programación y el análisis básico de datos.

En conclusión, el proyecto ayudó a reforzar los conocimientos adquiridos en clase y a entender cómo estos conceptos pueden aplicarse en la construcción de sistemas simples pero funcionales.